

OPERATING SYSTEMS

UE25CS242A

Multicore Programming, Multithreading Models and Thread Libraries

Dr. Anitha K
Department of Computer Science and Engineering

Acknowledgement, Prof. Pavan A C

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course



- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources:

1. “Operating System Concepts”, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne 10th Edition, John Wiley & Sons, Global Edition, 2023.
2. “Advanced Programming in the Unix Environment”, Richard Stevens and Stephen A Rago, Pearson, 3rd edition, 2017.
3. Some conceptual text and diagram from “Operating Systems, Internals and Design Principles”, William Stallings, 9th Edition, Pearson, 2018.

OPERATING SYSTEMS

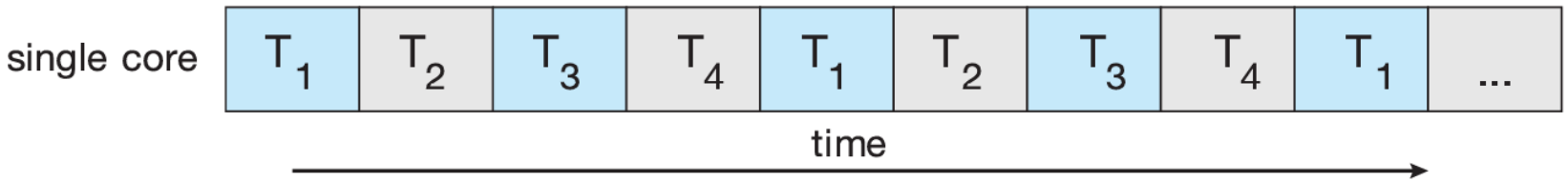
Multicore Programming , Multithreading Models and Thread Libraries

Dr. Anitha K
Department of Computer Science and Engineering

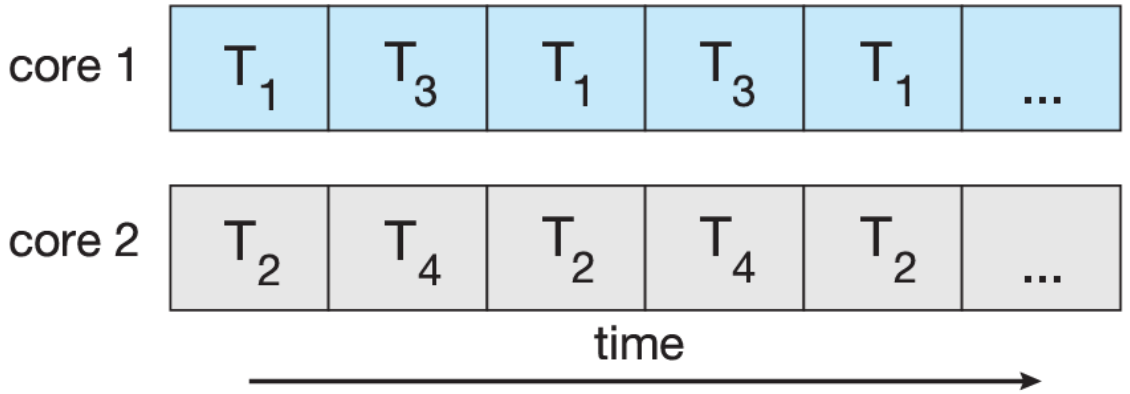
Multicore Programming



- Earlier, single-CPU systems evolved into multi-CPU systems.
- A later, design is to place multiple computing cores on a single processing chip where each core appears as a separate CPU to the operating system . such systems are referred as **Multicore.**
- On a system with a **single computing core**, the concurrency means the processing core is capable of executing only one thread at a time.
- On a system with **multiple cores**, however, concurrency means that some threads can run in parallel.



Concurrent execution on a single-core system.



Parallel execution on a multicore system.

Multicore Programming



- Multicore, and multithreaded programming provides a mechanism for more efficient use of these multiple computing cores and improved concurrency.
- On a system with a single computing core, concurrency merely means that the execution of the threads will be interleaved over time , because the processing core is capable of executing only one thread at a time.
- On a system with multiple cores, however, concurrency means that some threads can run in parallel, because the system can assign a separate thread to each core

Multicore Programming



five areas present challenges in programming for multicore systems:

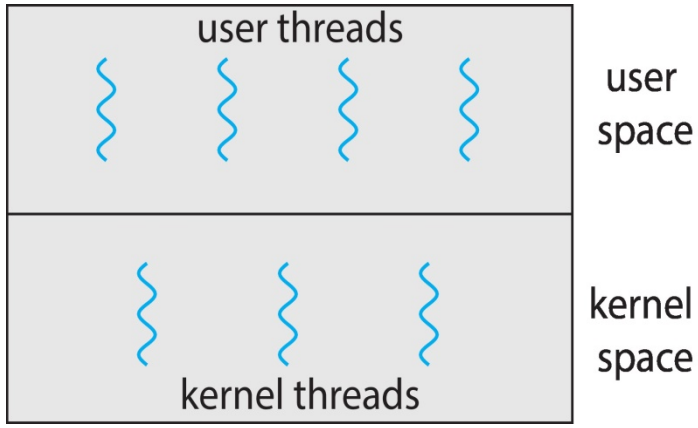
- Identifying tasks- tasks are independent of one another and thus can run in parallel on individual cores.
- Balance-programmers must also ensure that the tasks perform equal work of equal value
- Data splitting- applications are divided into separate tasks, the data accessed and manipulated by the tasks must be divided to run on separate cores.
- Data dependency- When one task depends on data from another, programmers must ensure that the execution of the tasks is synchronized to accommodate the data dependency.
- Testing and debugging- Testing and debugging concurrent programs is inherently more difficult than testing and debugging single-threaded applications.



- Types of Parallelism
- In general, there are two types of parallelism: data parallelism and task parallelism.
- Data parallelism focuses on distributing subsets of the same data across multiple computing cores and performing the same operation on each core.
- Task parallelism involves distributing not data but tasks (threads) across multiple computing cores.
- Each thread is performing a unique operation. Different threads may be operating on the same data, or they may be operating on different data.

User Threads and Kernel Threads

- **User threads** - management done by user-level threads library.
- Three primary thread libraries:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- **Kernel threads** - Supported by the Kernel.
- Examples – virtually all general purpose operating systems, including:
  - Windows
  - Solaris
  - Linux
  - Tru64 UNIX
  - Mac OS X



OPERATING SYSTEMS

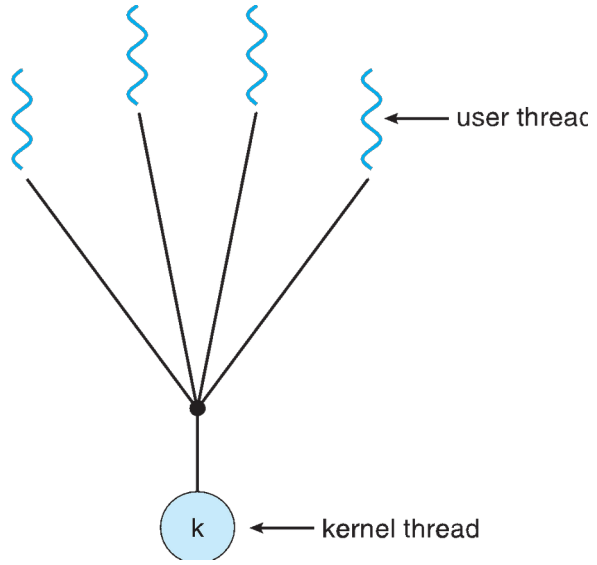
Multithreading Models

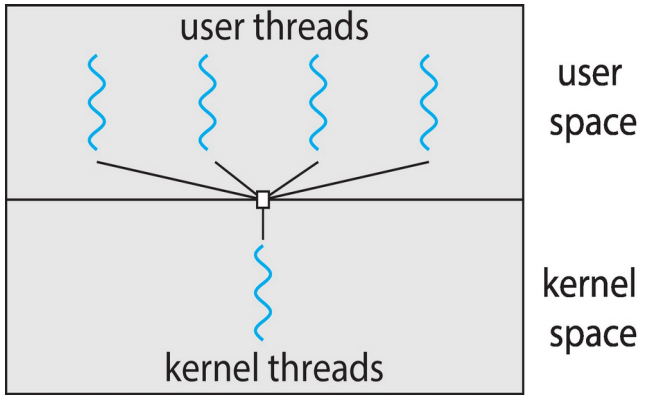
- Many-to-One
- One-to-One
- Many-to-Many

PES UNIVERSITY
CELEBRATING 50 YEARS

Many-to-One

- Many user-level threads mapped to single kernel thread.
- One thread blocking causes all to block.
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time.
- Few systems currently use this model.
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**

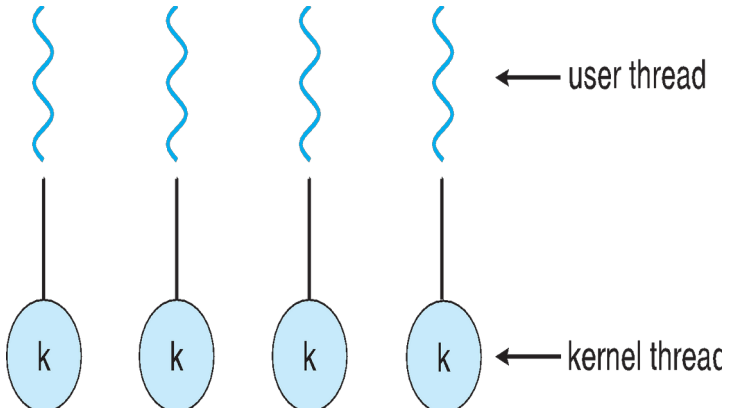


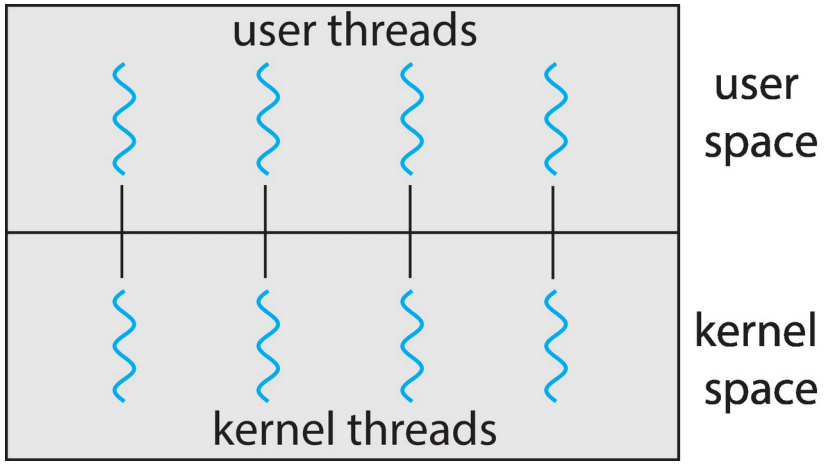


PES UNIVERSITY
CELEBRATING 50 YEARS

One-to-One

- Each user-level thread maps to kernel thread.
- Creating a user-level thread creates a kernel thread.
- More concurrency than many-to-one.
- Number of threads per process sometimes restricted due to overhead.
- Examples
  - Windows
  - Linux
  - Solaris 9 and later



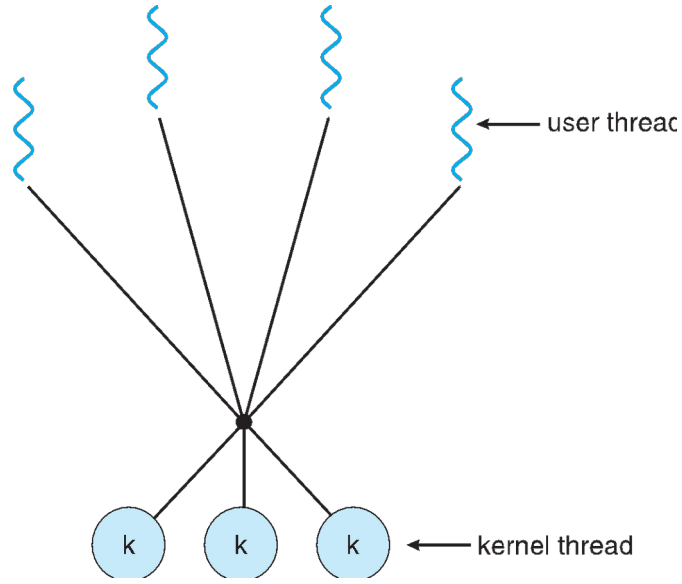


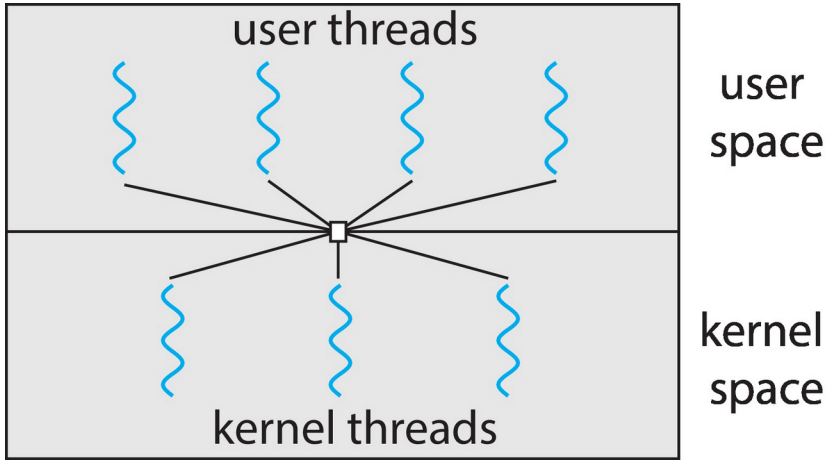
PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads.
- Allows the operating system to create a sufficient number of kernel threads.
- Solaris prior to version 9.
- Windows with the *ThreadFiber* package.

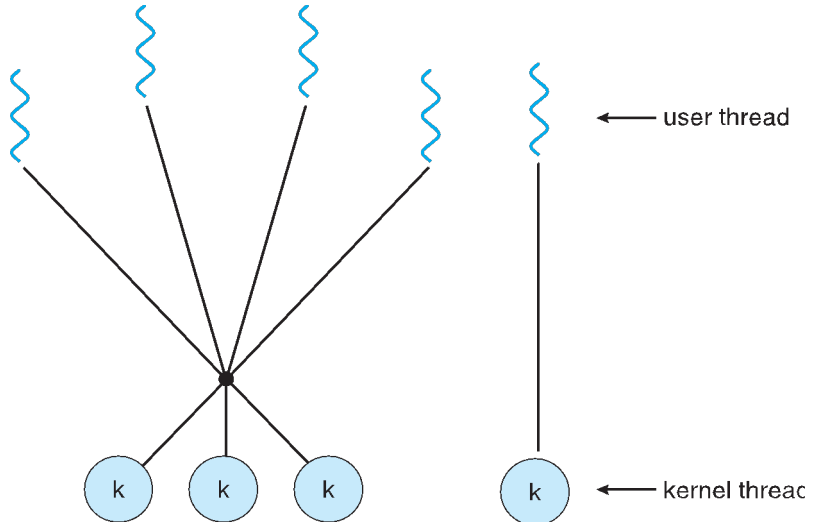


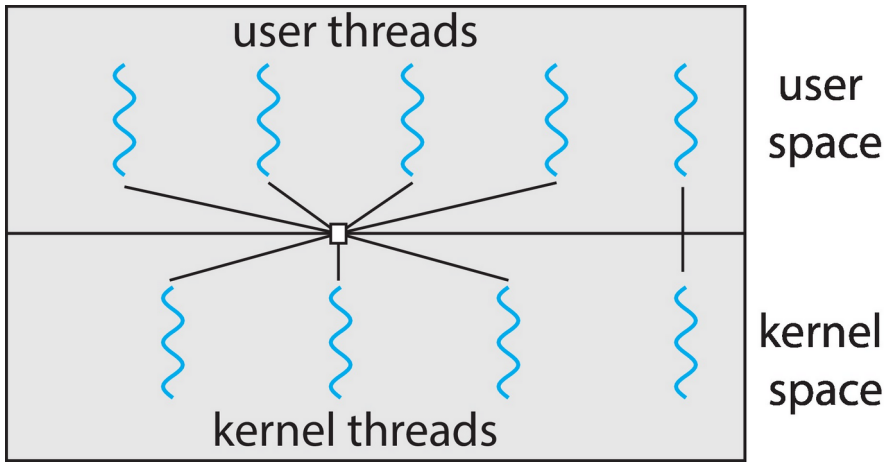


OPERATING SYSTEMS

Two-level Model

- Similar to M:M, except that it allows a user thread to be **bound** to kernel thread.
- Examples
  - IRIX
  - HP-UX
  - Tru64 UNIX
  - Solaris 8 and earlier





© copyright Silberschatz, Galvin and Gagne, 2013

Pthreads

- May be provided either as user-level or kernel-level.
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization.
- ***Specification***, not ***implementation.***
- API specifies behavior of the thread library, implementation is up to development of the library.
- Common in UNIX operating systems (Solaris, Linux, Mac OS X).

PES UNIVERSITY
CELEBRATING 50 YEARS

Pthreads: Thread Creation

- pthread_create: This is the primary function used to spawn a new flow of execution. Requires a function pointer (the "runner") for start routine.

int pthread_create(pthread_t ***thread***, const pthread_attr_t ***attr***, void *(***start_routine***) (void *), void ***arg***);

  - **thread**: A pointer to a pthread_t variable. This serves as the "handle" or ID for the thread once it is created.
  - **attr**: A pointer to an attribute structure.
  - **start_routine**: This is a function pointer. It points to the function the thread should execute. This function must take a void * as an argument and return a void *.
  - **arg**: The data you want to pass to the thread. Since it is a void *, you can pass a pointer to a primitive, a struct, or an array.

- Common mistake: passing a local variable's address to pthread_create.
- If the creating function returns before the thread starts, that local variable's address (on the stack) becomes invalid.
- The Solution: Always pass data that is either Global, on the Heap (using malloc), or in the main function stack which is guaranteed to exist for the duration of the program.

Pthreads: Synchronizing Execution



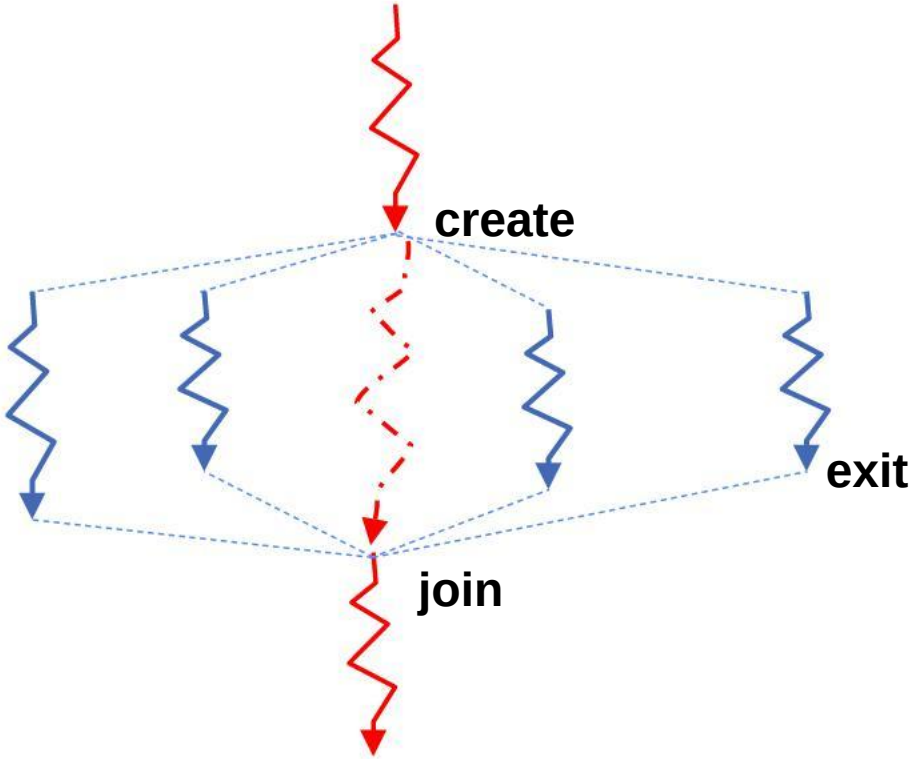
- In a multithreaded program, the main thread often needs to wait for its worker threads to finish their tasks before it can aggregate results or exit.

```
int pthread_join(pthread_t thread, void **retval);
```

- **retval**: Allows the caller to receive a return value from the thread that just finished.

- This function is blocking. If the target thread is still running, the caller (usually the main thread) will sleep until the target thread terminates.
- Cleanup: Joining is essential. If a thread is not "detached" and not "joined," it can become a Zombie Thread, consuming system resources even after it has finished its work.

Pthreads: Fork-Join Pattern



- Main thread *creates* (forks) collection of sub-threads passing them args to work on…
- … and then *joins* with them, collecting results.

Pthreads: Explicit Termination



- While a thread normally exit when its "runner" function finishes, you can terminate it prematurely using pthread_exit.

```
void pthread_exit(void *retval);
```

- Note: If you call this in the main() thread, the process will stay alive until all other threads have finished.
- If you simply call exit() or return from main(), the entire process including all its threads will be killed immediately.

OPERATING SYSTEMS

Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   if (argc != 2) {
      fprintf(stderr,"usage: a.out <integer value>\n");
      return -1;
   }
   if (atoi(argv[1]) < 0) {
      fprintf(stderr,"%d must be >= 0\n",atoi(argv[1]));
      return -1;
   }
```

PES UNIVERSITY
CELEBRATING 50 YEARS

© copyright Silberschatz, Galvin and Gagne, 2013

OPERATING SYSTEMS

Pthreads Example

```
    /* get the default attributes */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid,&attr,runner,argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid,NULL);

    printf("sum = %d\n",sum);
}

/* The thread will begin control in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

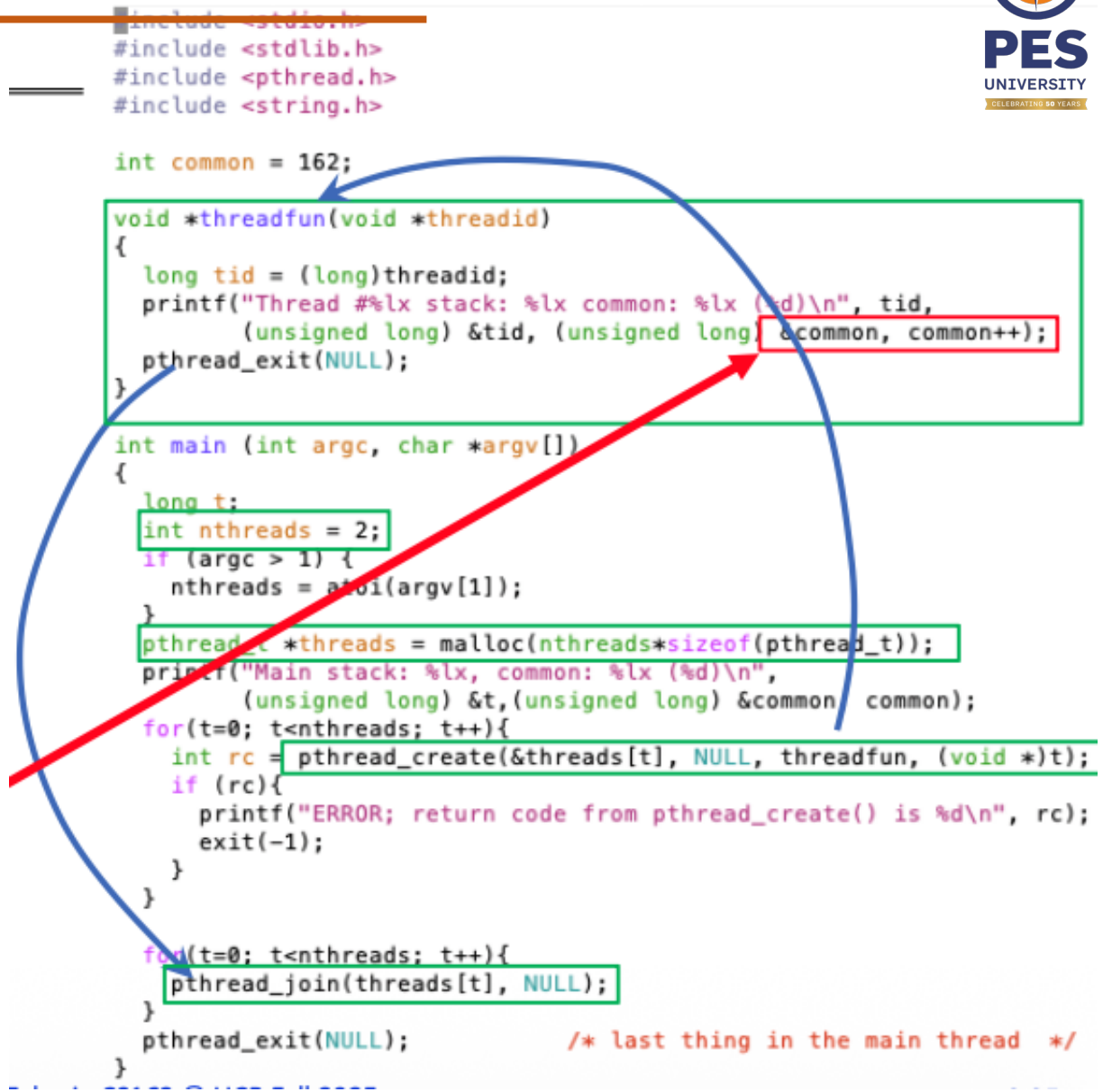
© copyright Silberschatz, Galvin and Gagne, 2013

PES UNIVERSITY
CELEBRATING 50 YEARS

Pthreads Example

- How many threads are in this program?
- What function does each thread run?

- Does the main thread join with the threads in the same order that they were created?
  - Yes: Loop calls Join in thread order
- Do the threads exit in the same order they were created?
  - No: Depends on scheduling order!
- Would the result change if run again?
  - Yes: Depends on scheduling order!
- Is this code safe/correct???
- No – threads share a variable that is used without locking and there is a race condition!



```
#include <stdlib.h>
#include <pthread.h>
#include <string.h>

int common = 162;

void *threadfun(void *threadid)
{
  long tid = (long)threadid;
  printf("Thread #%lx stack: %lx common: %lx (%d)\n", tid,
         (unsigned long) &tid, (unsigned long) &common, common++);
  pthread_exit(NULL);
}

int main (int argc, char *argv[])
{
  long t;
  int nthreads = 2;
  if (argc > 1) {
    nthreads = atoi(argv[1]);
  }
  pthread_t *threads = malloc(nthreads*sizeof(pthread_t));
  printf("Main stack: %lx, common: %lx (%d)\n",
         (unsigned long) &t,(unsigned long) &common, common);
  for(t=0; t<nthreads; t++){
    int rc = pthread_create(&threads[t], NULL, threadfun, (void *)t);
    if (rc){
      printf("ERROR; return code from pthread_create() is %d\n", rc);
      exit(-1);
    }
  }

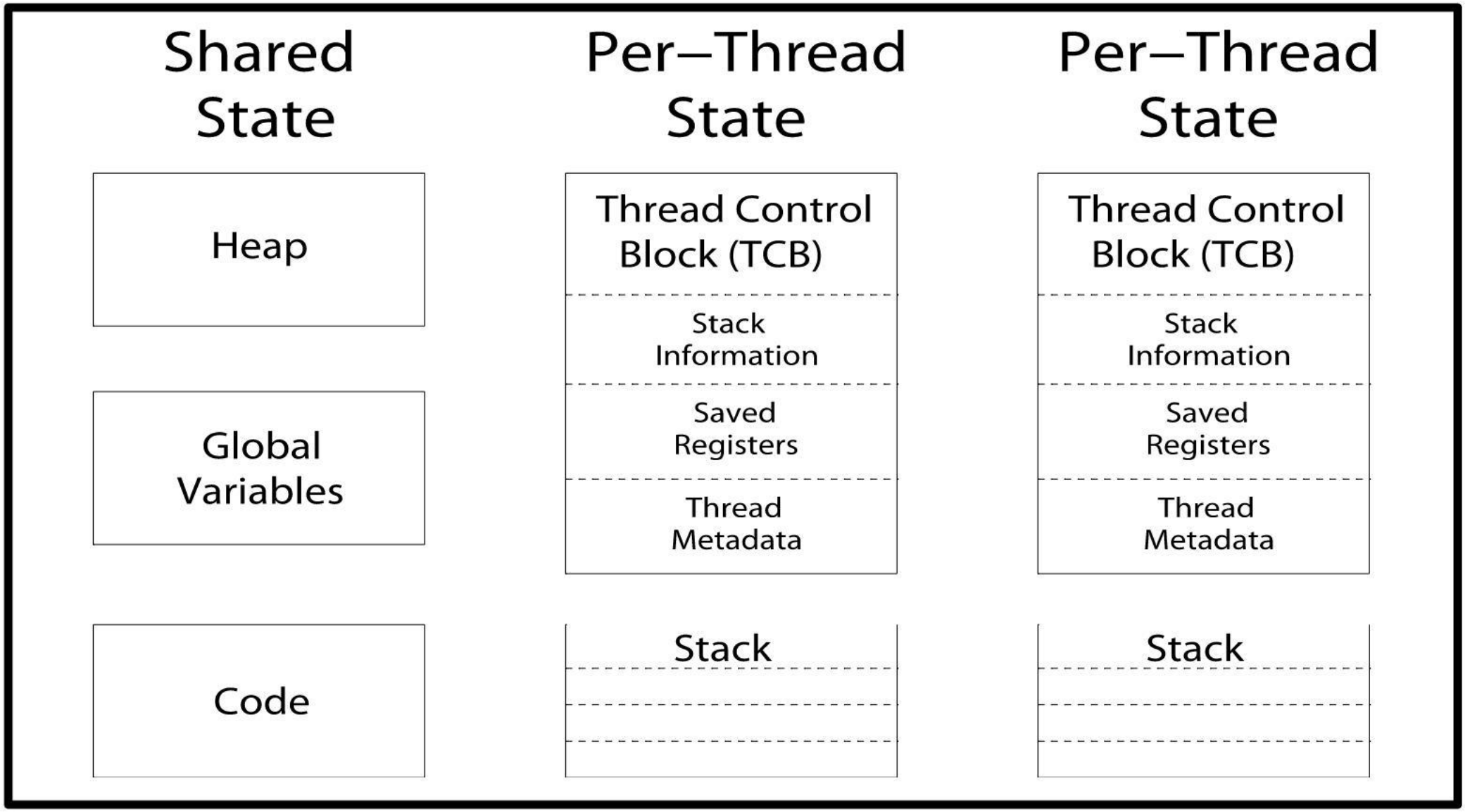
  for(t=0; t<nthreads; t++){
    pthread_join(threads[t], NULL);
  }
  pthread_exit(NULL);          /* last thing in the main thread  */
}
```

Thread States



- State shared by all threads in process/address space
  - Content of memory (global variables, heap)
  - I/O state (file descriptors, network connections, etc)
- State "private" to each thread
  - Kept in TCB -> Thread Control Block
  - CPU registers (including, program counter)
  - Execution stack
- Execution Stack
  - Parameters, temporary variables
  - Return PCs are kept while called procedures are executing

Shared vs. Per-Thread State



Thread Scheduling



- Many-to-one and many-to-many models, thread library schedules user-level threads to run on CPU through kernel.
  - Known as **process-contention scope (PCS)** since scheduling competition is within the process.
  - Typically done via priority set by programmer.
- Kernel thread scheduled onto available CPU is **system-contention scope (SCS)** – competition among all threads in system

Pthread Scheduling



- API allows specifying either PCS or SCS during thread creation
  - PTHREAD_SCOPE_PROCESS schedules threads using PCS scheduling
  - PTHREAD_SCOPE_SYSTEM schedules threads using SCS scheduling
- Can be limited by OS – Linux and Mac OS X only allow PTHREAD_SCOPE_SYSTEM

OPERATING SYSTEMS

Pthread Scheduling API




```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[]) {
   int i, scope;
   pthread_t tid[NUM THREADS];
   pthread_attr_t attr;
   /* get the default attributes */
   pthread_attr_init(&attr);
   /* first inquire on the current scope */
   if (pthread_attr_getscope(&attr, &scope) != 0)
      fprintf(stderr, "Unable to get scheduling scope\n");
   else {
      if (scope == PTHREAD_SCOPE_PROCESS)
         printf("PTHREAD_SCOPE_PROCESS");
      else if (scope == PTHREAD_SCOPE_SYSTEM)
         printf("PTHREAD_SCOPE_SYSTEM");
      else
         fprintf(stderr, "Illegal scope value.\n");
   }
```

OPERATING SYSTEMS

Pthread Scheduling API




```
/* set the scheduling algorithm to PCS or SCS */
  pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
  /* create the threads */
  for (i = 0; i < NUM_THREADS; i++)
     pthread_create(&tid[i],&attr,runner,NULL);
  /* now join on each thread */
  for (i = 0; i < NUM_THREADS; i++)
     pthread_join(tid[i], NULL);
}
/* Each thread will begin control in this function */
void *runner(void *param)
{
  /* do some work ... */
  pthread_exit(0);
}
```

Advanced Thread Management

PES UNIVERSITY
CELEBRATING 50 YEARS

- **Thread Pools:** Creating threads is faster than processes, but doing it thousands of times for small tasks is still wasteful.
- A Thread Pool creates a fixed number of threads at startup that "sit and wait" for work, preventing resource exhaustion.
- **Thread Cancellation:** Terminating a thread before it finishes.
- **Asynchronous:** Terminate immediately (Dangerous! Might leave a Mutex locked).
- **Deferred:** The thread periodically checks if it should exit (Safer).



THANK YOU

Dr. Anitha K

Department of Computer Science and Engineering

anithak@pes.edu